Tier 2A/2B: the optional executable-model contract

a validated declaration facet, and a port axis independent of the sign

Created-at: 2026-09-06 03:52 +02:00

Entry state, measured after fetch. Core `main@4acba44`, clean, **fully green**: 134 + 254 + 2 + 5 = 395 passed, 0 failed, 7 ignored; `clippy -p hymeko_core -all-targets -D warnings` 0; `fmt -check` 0. The 2026-09-05 baseline failure `v1_corpus_digest_is_unchanged` **now passes** — it was resolved upstream, and that earlier "not green" record is superseded. Provider `hymeko@1032a2d`. Downstream Nagare witnesses on this core: `merkle_digest_soundness` 14/14 with 0 ignored, `canonical_mlp_witness` 2/2, `contract_consumer_gate` 10/10.

The gap, re-reproduced on this tip. Of the eleven ownership rows, exactly one is `CanonicalOwned` (the semantic model digest, upstream). Rows 2, 4, 7 and 9 are `Missing`; 1, 3, 5, 6, 8, 10 are `CarrierOnly`. Verified by direct inspection: the IR entity set in `ir/ir.rs` is unchanged, no `Role`/`Shape`/`DType`/ direction type exists anywhere in `hymeko_core/src`, `ir::coherence` contains no annotation inspection, and `tensor/compute/mod.rs:26` still declares `DType` deliberately absent.

1 The design question that comes first

Most HyMeKo declarations are not tensors or operators. The tracked corpus describes service networks, robotics meta-graphs and P-graph structures; the 111 `.hymeko` fixtures under `tests/` contain no neural model. A mandatory `role`/`shape`/`dtype` on every `DeclNode` would therefore be wrong on its face: it would force a meaningless answer onto every existing declaration and move every existing digest.

The contract is consequently an **optional facet**, not a widening of the universal declaration:

`DeclNode.model: Option<ModelFacet>`	absent (`None`) on every declaration that is not part of an executable model — which is every declaration in the current corpus
`ModelFacet::Tensor`	`{role, shape, dtype}`: the declaration is a tensor
`ModelFacet::Operator`	`{identity}`: the declaration is a typed operator

A document is an *executable model* exactly when at least one declaration carries a facet. Only then does the completeness validator engage. This is what keeps the general IR general, and it is the reason the change can be additive rather than a migration.

Annotation tags are not the authority. Where a facet is present, the typed fields are authoritative and the tags are decoration. The acceptance gate is adversarial: an executable model whose tags are replaced by a meaningless word must still validate and still digest identically, while removing a typed field must fail. That is the inverse of today's `core_accepts_a_model_whose_tag_vocabulary_is_meaningless`, which currently passes and must, after 2A, fail for a model that declares facets.

2 Compatibility, proven before the first edit

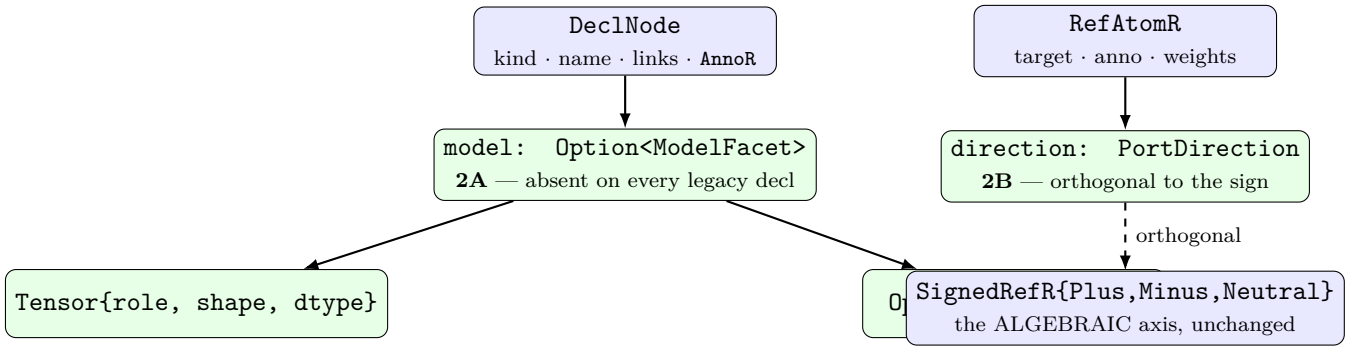
Three questions decide whether this boundary may proceed at all.

Question	Evidence gathered at entry
Do absolute V2 digest goldens exist?	**No.** `tests/unified_ir/verification/semantic_digest.rs` contains **zero** 64-hex constants; every assertion is relative (moves / does not move). The only absolute goldens are v1-family: `KAT_CORPUS_DIGEST`, the four `frozen_hash.rs` KATs, and the unrelated `sparse_workload_kat`. **So the algorithm version may be raised, and v1 must not be touched.**
Is the CBOR wire serde-derived?	**Yes** — `ciborium::ser::into_writer(&payload)` over `CborPayload`, which already carries an explicit `schema_version: u16` that rejects missing and unsupported versions. An additive field with `#[serde(default)]` therefore decodes every existing payload unchanged.
How many `RefAtomR` construction sites?	**Five** literal sites in `hymeko_core/src` (`ir/ir.rs`, `ir/incidence.rs`, `resolution/resolve.rs`, `writers/cbor_writer/tests.rs`). The 2B inventory is bounded and enumerable before any signature moves.

Versioning. `SEMANTIC_DIGEST_ALGO_VERSION` 2 → 3, because the covered set grows. `canonical_program_hash` v1 is **not touched**: no signature, no byte, no `algo_version`, so `KAT_CORPUS_DIGEST` and the frozen KATs hold unchanged. The CBOR `schema_version` rises only if the encoded shape changes in a way old readers cannot ignore; the additive-optional form is chosen precisely so it does not.

Legacy digests. A declaration with `model: None` contributes one domain-tagged absence marker. Every V2 digest therefore moves once, at the version bump — which is sound because no artifact pins a V2 value — and never again for a legacy document.

3 Architecture



Two axes, never derived from one another.
A legacy incidence gets `Unspecified`, never an invented direction.

4 Tier 2A — validated model declaration contract

Owner. `hymeko_core::ir` for the types, `ir::coherence` for validation, `ir::semantic_digest` for coverage. No runtime tensor framework enters core: a shape is a list of extents and a dtype is a closed enum, neither carries data nor arithmetic.

Shape. Rank-general by construction (`Vec` of extents, symbolic or static), so the later CV work is not blocked, while Slice 1 exercises only the small 1-D/2-D MLP shapes.

Acceptance gates.

1. witness rows 1–6, 9 and 10 become `CanonicalOwned`;
2. an executable model whose tags are replaced by a meaningless word is **rejected** when a required typed field is absent, and **digest-identical** when only the tags differ;
3. a mutation of `role`, `shape`, `dtype` or operator identity moves the semantic digest — each with an observation floor proving the field actually changed;
4. serialize → parse preserves the typed contract and the digest;
5. the existing corpus decodes unchanged and its v1 digest is byte-identical;
6. a malformed shape, an unknown dtype and an incomplete facet are each rejected for the stated reason, by a negative control that fails before the fix.

5 Tier 2B — port direction, orthogonal to the sign

The rule. `Plus/Minus/Neutral` is the *algebraic* sign of a signed hypergraph incidence. It is never read as data-flow direction. `PortDirection` is a separate typed axis on `RefAtomR` with an explicit `Unspecified` for legacy incidences — a legacy edge is never given an invented direction.

Inventory before any signature moves. Five `RefAtomR` literal sites in core, plus the parser/lowering path (`resolution::resolve`) and the consumers (`semantic_digest`, `hash_pass`, `cbor_writer::validate`, `ir::incidence`). Each is enumerated and classified before the field is added.

Acceptance gates.

1. the witness's data-flow-direction row becomes `CanonicalOwned`;
2. same sign, changed direction ⇒ the digest moves;
3. same direction, changed sign ⇒ the digest moves, and the two are detected as *separate* mutations;
4. swapping input and output raises a coherence error;
5. round-trip preserves both axes field-by-field;
6. existing corpus behaviour is explicit: every legacy incidence is `Unspecified`, asserted.

6 Order, and the stop conditions

2A first, in the smallest coherent slice, with its narrow unit and mutation controls, then the core integration/serialization/digest gates, then a diff audit. **2B starts only if 2A is green.** The full core gate and the canonical MLP witness re-run close the boundary. No WGPU, no Nagare `ExecutionPlan`, no CV, no MCU, no dataset campaign, no dependency, no `CORE.YAML` schema change beyond the approved-edit ledger entry.

Halt immediately if: a wire-format change cannot be proven compatible; an unknown consumer appears; a gate passes only vacuously; 2B turns out to be derivable from the sign alone (which would mean the axes are not orthogonal and the design is wrong); or upstream drift changes the gap.

Rollback. Every addition is an optional field plus new types. Removing `ModelFacet`, the `direction` field, their digest arms and their tests restores the prior system exactly; v1 is untouched throughout, so no recorded artifact depends on the change.